SQL: подзапросы и оконные функции

Два урока с разбором запросов, наглядными схемами и практикой на одной базе данных.

Этот материал поможет перейти от выборки отдельных строк к запросам, которые сравнивают данные между собой: находят заказы выше среднего, выбирают последние покупки и считают накопительные суммы.

> **Что нужно знать заранее**
>
> SELECT, FROM, WHERE, JOIN, GROUP BY, HAVING и агрегатные функции COUNT, SUM, AVG. Если группировка ещё вызывает вопросы, сначала повторите её на простой таблице.

Две идеи, которые стоит запомнить

Подзапрос	Оконная функция
Вложенный SELECT даёт значение, набор значений или таблицу для другого запроса.	Вычисление OVER (...) добавляет результат к каждой строке выбранного набора.
«Выбери заказы дороже средней покупки».	«Покажи каждый заказ и сумму покупок его клиента».

Как устроен конспект

Раздел	Страницы
Учебная база и SQL для её создания	2-3
Урок 1. Подзапросы: теория и примеры	4-8
Практика: 3 задачи на подзапросы	9
Урок 2. Оконные функции: теория и примеры	10-16
Практика: 3 задачи на оконные функции	17
Вопросы для самопроверки и источники	18

Примеры написаны для PostgreSQL. В других СУБД синтаксис и отдельные возможности могут отличаться. Все данные учебные; суммы указаны в рублях. В результатах средние при необходимости округлены до двух знаков.

ОБЩАЯ БАЗА ДЛЯ ДВУХ УРОКОВ

Интернет-магазин в двух таблицах

Один клиент может сделать несколько заказов. Каждый заказ относится к одному клиенту.

`customers.id` связан с `orders.customer_id`. Идентификатор `orders.id` уникален: он поможет однозначно упорядочить заказы с одинаковой датой.

customers - клиенты

id	name	city
1	Анна	Москва
2	Борис	Казань
3	Вера	Москва
4	Глеб	Омск
5	Дина	Казань

orders - заказы

id	customer_id	order_date	amount	status
101	1	2026-03-01	1 000	paid
102	2	2026-03-01	1 000	paid
103	1	2026-03-03	2 000	paid
104	3	2026-03-02	2 500	paid
105	2	2026-03-04	1 000	paid
106	1	2026-03-03	2 000	paid
107	4	2026-03-05	5 000	cancelled
108	3	2026-03-06	3 500	paid
109	1	2026-03-07	3 000	paid
110	2	2026-03-08	2 000	paid

Правило для всех примеров и задач

В расчётах оплат учитываем только status = 'paid'. Заказ 107 отменён. У Глеба есть только отменённый заказ, у Дины нет заказов вообще. У Анны заказы 103 и 106 совпадают по дате и сумме.

Создайте учебную базу

Выполните код один раз в пустой схеме PostgreSQL. Затем запускайте примеры и свои решения.

SQL / СОЗДАНИЕ ТАБЛИЦ И ДАННЫХ

```sql
CREATE TABLE customers (
    id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    city TEXT NOT NULL
);

CREATE TABLE orders (
    id INTEGER PRIMARY KEY,
    customer_id INTEGER NOT NULL REFERENCES customers(id),
    order_date DATE NOT NULL,
    amount NUMERIC(10, 2) NOT NULL CHECK (amount > 0),
    status TEXT NOT NULL CHECK (status IN ('paid', 'cancelled'))
);

INSERT INTO customers (id, name, city) VALUES
    (1, 'Анна',  'Москва'),
    (2, 'Борис', 'Казань'),
    (3, 'Вера',  'Москва'),
    (4, 'Глеб',  'Омск'),
    (5, 'Дина',  'Казань');

INSERT INTO orders
    (id, customer_id, order_date, amount, status) VALUES
    (101, 1, '2026-03-01', 1000, 'paid'),
    (102, 2, '2026-03-01', 1000, 'paid'),
    (103, 1, '2026-03-03', 2000, 'paid'),
    (104, 3, '2026-03-02', 2500, 'paid'),
    (105, 2, '2026-03-04', 1000, 'paid'),
    (106, 1, '2026-03-03', 2000, 'paid'),
    (107, 4, '2026-03-05', 5000, 'cancelled'),
    (108, 3, '2026-03-06', 3500, 'paid'),
    (109, 1, '2026-03-07', 3000, 'paid'),
    (110, 2, '2026-03-08', 2000, 'paid');
```

Проверка загрузки

В customers должно быть 5 строк, в orders - 10. Оплаченных заказов 9; их общая сумма 18 000, средняя сумма заказа 2 000.

id - первичный ключ. customer_id - внешний ключ. NUMERIC(10, 2) хранит суммы точно, с двумя десятичными знаками. NOT NULL запрещает отсутствие значения. Правила CHECK ограничивают допустимые суммы и статусы.

Сначала задайте вложенный вопрос

Подзапрос - это SELECT внутри другого SQL-запроса. Вложенный результат помогает решить внешнюю задачу.

Вопрос «какие заказы дороже среднего?» удобно разделить на два: чему равна средняя сумма оплаченного заказа и какие оплаченные заказы превышают это значение.



ПРИМЕР 1 / СКАЛЯРНЫЙ ПОДЗАПРОС

```sql
SELECT id, customer_id, amount
FROM orders
WHERE status = 'paid'
  AND amount > (
      SELECT AVG(amount)
      FROM orders
      WHERE status = 'paid'
  )
ORDER BY id;
```

id	customer_id	amount
104	3	2 500
108	3	3 500
109	1	3 000

Здесь вложенный SELECT возвращает **одно значение** - 2 000. Такой подзапрос называется **скалярным**. Его можно использовать там, где ожидается обычное значение: в сравнении, вычислении или списке SELECT. [1]

Сколько строк может вернуть скалярный подзапрос?

Один столбец и не более одной строки. Нет строк - получится NULL. Несколько строк - ошибка PostgreSQL. AVG без GROUP BY возвращает одну строку; если подходящих значений нет, её значение равно NULL.

Схема показывает способ рассуждения. Фактический порядок и способ выполнения выбирает оптимизатор: он может преобразовать запрос.

IN, EXISTS и NOT EXISTS

Выбирайте оператор по вопросу: «входит в набор?» или «есть хотя бы одна подходящая строка?» [2]

IN: входит ли значение в результат подзапроса?

ПРИМЕР 2 / КЛИЕНТЫ С ОПЛАТОЙ ОТ 3 000

```sql
SELECT id, name
FROM customers
WHERE id IN (
    SELECT customer_id
    FROM orders
    WHERE status = 'paid' AND amount >= 3000
)
ORDER BY id;
```

Результат: Анна (id = 1) и Вера (id = 3). Подзапрос возвращает один столбец, но может вернуть много строк. Повторы значений внутри IN не создают повторов строк внешней таблицы.

EXISTS: есть ли совпадение для текущего клиента?

ПРИМЕР 3 / ЕСТЬ ХОТЯ БЫ ОДНА ОПЛАТА

```sql
SELECT c.id, c.name
FROM customers AS c
WHERE EXISTS (
    SELECT 1
    FROM orders AS o
    WHERE o.customer_id = c.id AND o.status = 'paid'
)
ORDER BY c.id;
```

Результат: Анна, Борис и Вера. SELECT 1 - принятое обозначение: важен сам факт наличия строки, а не значение 1. Условие o.customer_id = c.id связывает заказ с текущим клиентом.

NOT EXISTS: нет ни одного совпадения

Замените EXISTS в примере 3 на **NOT EXISTS**: получите Глеба и Дину. У Глеба есть заказ, но он отменён; у Дины нет заказов. У обоих отсутствуют строки, удовлетворяющие **всем** условиям подзапроса.

Почему не всегда достаточно JOIN?

JOIN с заказами может повторить клиента по числу подходящих заказов. EXISTS только проверяет условие для уже рассматриваемой строки клиента. Для проверки «есть ли» это часто самая ясная запись.

Подзапрос зависит от внешней строки

Коррелированный подзапрос обращается к столбцу внешнего запроса. Его результат может меняться от строки к строке.

Теперь ищем оплаченные заказы, которые дороже средней покупки **своего клиента**. Общая средняя по магазину уже не подходит: для каждого клиента нужна собственная величина.

ПРИМЕР 4 / СРАВНЕНИЕ СО СВОЕЙ СРЕДНЕЙ

```sql
SELECT o.id, o.customer_id, o.amount
FROM orders AS o
WHERE o.status = 'paid'
  AND o.amount > (
      SELECT AVG(o2.amount)
      FROM orders AS o2
      WHERE o2.customer_id = o.customer_id
        AND o2.status = 'paid'
  )
ORDER BY o.id;
```

Ключевая строка - `o2.customer_id = o.customer_id`. Алиас **o2** обозначает строки внутри подзапроса, **o** - текущую строку снаружи. [1, 2]

Клиент	Оплаченные суммы	Средняя	Выше средней
Анна	1 000; 2 000; 2 000; 3 000	2 000	109
Борис	1 000; 1 000; 2 000	1 333,33	110
Вера	2 500; 3 500	3 000	108

Результат: заказы **108, 109 и 110**. Например, 104 стоит 2 500: это выше средней по магазину, но ниже средней Веры. Поэтому в примере 1 он был, а здесь его нет.

Модель понимания и реальное выполнение

Для понимания можно мысленно подставлять внешнее значение в подзапрос для каждой строки. Это не обещание, что СУБД физически запускает его заново каждый раз: план зависит от запроса и оптимизатора.

Практический приём: сначала подставьте конкретный customer_id и запустите внутренний SELECT отдельно. Затем замените число ссылкой на внешний алиас.

В FROM - таблица, в SELECT - значение

Форма результата определяет место использования подзапроса.

Подзапрос в FROM: промежуточная таблица

Посчитаем среднюю сумму всех оплат **на одного платившего клиента**. Сначала получим итог каждого клиента, затем усредним эти итоги.

ПРИМЕР 5 / СРЕДНЕЕ ОТ ИТОГОВ

```
SELECT ROUND(AVG(t.total_paid), 2) AS avg_customer_total
FROM (
    SELECT customer_id, SUM(amount) AS total_paid
    FROM orders
    WHERE status = 'paid'
    GROUP BY customer_id
) AS t;
```

Промежуточная таблица t	total_paid
customer_id = 1	8 000
customer_id = 2	4 000
customer_id = 3	6 000

Результат внешнего SELECT: 6 000. Это (8 000 + 4 000 + 6 000) / 3. Клиенты без оплат здесь не участвуют. Средняя сумма отдельного заказа, 2 000, отвечает на другой вопрос.

Подзапрос в SELECT: дополнительный столбец

ПРИМЕР 6 / СКОЛЬКО ОПЛАТ У КЛИЕНТА

```
SELECT c.id, c.name,
       (SELECT COUNT(*)
        FROM orders AS o
        WHERE o.customer_id = c.id
          AND o.status = 'paid') AS paid_count
FROM customers AS c
ORDER BY c.id;
```

Получим: Анна - 4, Борис - 3, Вера - 2, Глеб - 0, Дина - 0. COUNT(*) без GROUP BY возвращает одну строку со значением 0, если подходящих заказов нет.

> **Проверяйте форму результата**
>
> В FROM можно вернуть несколько строк и столбцов. В этом конспекте всегда задаём алиас таблицы, например AS t: через него обращаемся к столбцам результата. В SELECT нужен скалярный результат. [1, 3]

NULL и другие подводные камни

В SQL NULL означает отсутствие или неизвестность значения. Сравнение с неизвестным не даёт обычное TRUE или FALSE.

Почему NOT IN может неожиданно убрать строки

МИНИ-ПРИМЕР / РЕЗУЛЬТАТ NULL

```
SELECT 5 NOT IN (1, 2, NULL) AS result;
```

Число 5 не равно 1 и 2, но его сравнение с NULL неизвестно. Поэтому всё выражение даёт NULL. В **WHERE проходят только строки с TRUE**; FALSE и NULL отбрасываются. В подзапросе внутри NOT IN возможен тот же эффект. [2]

Если нужно найти клиентов без оплат, удобно выразить это через отсутствие связанной строки:

ПРИМЕР 7 / КЛИЕНТЫ БЕЗ ОПЛАТ

```
SELECT c.id, c.name
FROM customers AS c
WHERE NOT EXISTS (
    SELECT 1
    FROM orders AS o
    WHERE o.customer_id = c.id AND o.status = 'paid'
)
ORDER BY c.id;
```

В нашей схеме orders.customer_id объявлен NOT NULL, поэтому именно в этом столбце такой ловушки нет. Но в другой таблице или вычисляемом подзапросе NULL возможен. Для проверки самого NULL используйте IS NULL, а не = NULL.

Как выбрать конструкцию

Нужный результат	Подход
Одно число для сравнения	Скалярный подзапрос
Проверка вхождения в набор	IN
Наличие / отсутствие подходящей строки	EXISTS / NOT EXISTS
Расчёт зависит от текущей строки	Коррелированный подзапрос
Промежуточный набор строк и столбцов	Подзапрос в FROM

Перед запуском: проверьте скобки, алиасы и фильтр paid на каждом нужном уровне. Не исправляйте ошибку «несколько строк» случайным LIMIT 1: сначала определите, нужен ли IN, агрегат или осмысленный выбор одной строки.

Три задачи на подзапросы

Используйте таблицы со страниц 2-3. Запросы должны вычислять значения из данных, без подстановки готовых средних и id.

1. Заказы дешевле среднего

Найдите оплаченные заказы, сумма которых **строго меньше** средней суммы всех оплаченных заказов. Используйте скалярный подзапрос в WHERE. Выведите id, customer_id, amount; отсортируйте по id.

Контроль: 3 строки - заказы 101, 102 и 105. У всех сумма 1 000. Заказы ровно на 2 000 в результат не входят.

2. Заказы есть, оплат нет

Найдите клиентов, у которых есть **хотя бы один заказ любого статуса**, но нет ни одного оплаченного заказа. Используйте EXISTS и NOT EXISTS. Выведите id, name; отсортируйте по id.

Контроль: одна строка - 4, Глеб. Дина не подходит: у неё нет заказов. Подсказка: это две независимые проверки для одного клиента.

3. Выше средней своего клиента

Найдите оплаченные заказы, сумма которых строго больше средней суммы оплаченных заказов **того же клиента**. Используйте коррелированный подзапрос. Выведите id заказа, имя клиента, amount и customer_avg. Среднюю округлите до 2 знаков только при выводе; сравнивайте с исходной средней. Сортировка - по id заказа.

id	name	amount	customer_avg
108	Вера	3 500	3 000
109	Анна	3 000	2 000
110	Борис	2 000	1 333,33

Как проверить решение

Убедитесь, что 107 исключён как отменённый, а средние считаются только по оплатам. Мысленно добавьте клиенту ещё одну оплату: запрос должен пересчитать ответ без изменения SQL.

Вычисляйте по группе, сохраняя строки

Оконная функция видит связанные строки и записывает результат рядом с текущей строкой. Её признак - OVER (...). [4]

GROUP BY объединяет строки в группы и выдаёт итог на группу. Оконная функция позволяет одновременно видеть **детали заказа** и **итог клиента**. Ниже два независимых результата для четырёх оплат Анны.



GROUP BY / ОДНА СТРОКА НА КЛИЕНТА

```
SELECT customer_id, SUM(amount) AS total_paid
FROM orders
WHERE status = 'paid'
GROUP BY customer_id
ORDER BY customer_id;
```

OVER / КАЖДЫЙ ОПЛАЧЕННЫЙ ЗАКАЗ ОСТАЁТСЯ

```
SELECT id, customer_id, amount,
       SUM(amount) OVER (
           PARTITION BY customer_id
       ) AS customer_total
FROM orders
WHERE status = 'paid'
ORDER BY customer_id, id;
```

Первый запрос возвращает 3 строки: по одной на платившего клиента. Второй - 9: по одной на оплаченный заказ. У Анны в каждой из 4 строк будет customer_total = 8 000.

Какие строки видит окно?

Окно работает с набором, полученным после FROM, WHERE, GROUP BY и HAVING этого уровня запроса. В наших примерах с фильтром paid отменённые заказы в расчёт не попадают.

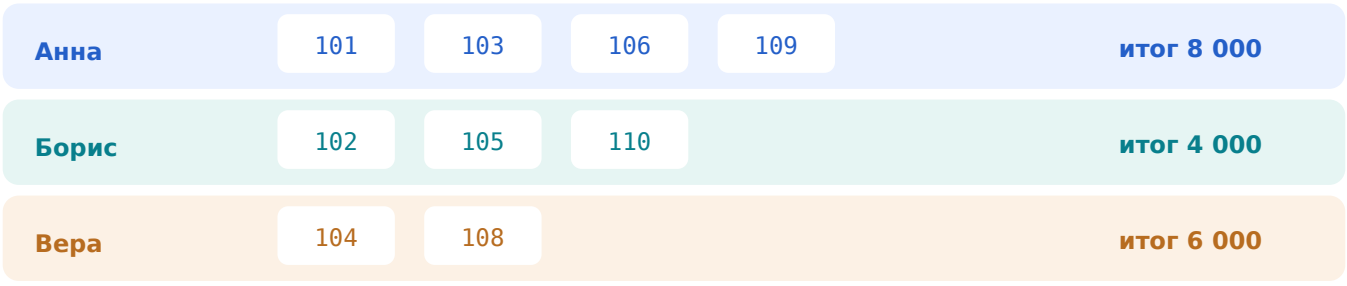
Раздел, порядок и рамка

У OVER три разных настройки. Они отвечают на три разных вопроса.

ШАБЛОН / ЗАПОЛНИТЕ НУЖНЫЕ ЧАСТИ

```
function(...) OVER (
    PARTITION BY ...
    ORDER BY ...
    ROWS BETWEEN ... AND ...
)
```

Часть	Вопрос	Пример
PARTITION BY	Для каких независимых групп считать?	Отдельно для каждого customer_id
ORDER BY внутри OVER	В каком порядке рассматривать строки?	По order_date, затем id
Рамка ROWS / RANGE	Какая часть раздела участвует в текущем расчёте?	От начала до текущей строки



Каждая цветная полоса - отдельный раздел PARTITION BY customer_id

Если **PARTITION BY не указан**, весь входной набор считается одним разделом. Например, `SUM(amount) OVER ()` вернёт общий итог 18 000 в каждой строке оплаченного заказа.

Если **ORDER BY внутри OVER не указан**, для SUM и AVG по умолчанию используется весь раздел. Например, `SUM(amount) OVER (PARTITION BY customer_id)` даёт сумму оплат клиента. [1]

> **Два ORDER BY выполняют разные роли**
>
> ORDER BY внутри OVER задаёт порядок вычисления. Последний ORDER BY всего SELECT задаёт порядок отображения результата. Один не заменяет другой. Для последовательности заказов используем order_date, id.

Итог клиента и доля заказа

SUM, AVG и COUNT могут работать как оконные функции: достаточно добавить OVER (...).

Найдём сумму оплат клиента и долю каждого заказа в этой сумме. Разделяем строки по клиенту; порядок здесь не нужен, потому что считаем **по всему разделу**.

ПРИМЕР 8 / ДОЛЯ В ОПЛАТАХ КЛИЕНТА

```
SELECT id, customer_id, amount,
       SUM(amount) OVER (
           PARTITION BY customer_id
       ) AS customer_total,
       ROUND(100.0 * amount / SUM(amount) OVER (
           PARTITION BY customer_id
       ), 2) AS share_pct
FROM orders
WHERE status = 'paid'
ORDER BY customer_id, order_date, id;
```

Фрагмент результата: Анна

id	amount	customer_total	share_pct
101	1 000	8 000	12,50
103	2 000	8 000	25,00
106	2 000	8 000	25,00
109	3 000	8 000	37,50

Заказ 109 на 3 000 составляет 37,5% всех оплат Анны: `100.0 * 3000 / 8000`. Итог 8 000 повторяется в каждой строке, но сам заказ не исчезает.

Выражение	Что добавит к каждой строке
AVG(amount) OVER (...)	Среднюю сумму в соответствующей рамке
COUNT(*) OVER (...)	Количество строк в соответствующей рамке
SUM(amount) OVER ()	Общую сумму всех строк входного набора

В нашей базе amount > 0, поэтому сумма непустого раздела больше нуля. В другой модели данных отдельно продумайте деление на ноль. Число 100.0 помогает избежать целочисленного деления при адаптации запроса.

ROW_NUMBER, RANK, DENSE_RANK

Все три функции используют порядок внутри OVER, но по-разному обрабатывают одинаковые значения. [5]

Функция	Что происходит при равенстве
ROW_NUMBER()	У каждой строки свой номер: 1, 2, 3, 4.
RANK()	У равных значений одно место; после них есть пропуск: 1, 2, 2, 4.
DENSE_RANK()	У равных значений одно место; пропусков нет: 1, 2, 2, 3.

ПРИМЕР 9 / ОПЛАТЫ АННЫ ПО УБЫВАНИЮ СУММЫ

```
SELECT id, amount,
       ROW_NUMBER() OVER (
           ORDER BY amount DESC, id
       ) AS row_num,
       RANK() OVER (ORDER BY amount DESC) AS rnk,
       DENSE_RANK() OVER (ORDER BY amount DESC) AS dense_rnk
FROM orders
WHERE status = 'paid' AND customer_id = 1
ORDER BY amount DESC, id;
```

id	amount	row_num	rnk	dense_rnk
109	3 000	1	1	1
103	2 000	2	2	2
106	2 000	3	2	2
101	1 000	4	4	3

Важный нюанс: что считать равенством

Для RANK и DENSE_RANK равенство определяется всеми полями ORDER BY внутри OVER. Если добавить туда уникальный id, равных строк уже не останется. Для мест по сумме оставляем amount, а для устойчивой нумерации ROW_NUMBER добавляем id.

Чтобы нумерация начиналась заново для каждого клиента, добавьте PARTITION BY customer_id. Условие row_num <= 2 выбирает не более двух строк; dense_rnk <= 2 - все строки с двумя наибольшими различными суммами. Способ фильтрации - на странице 16.

LAG и LEAD: сравните покупки

LAG берёт значение из предыдущей строки раздела, LEAD - из следующей. Направление задаётся ORDER BY внутри OVER. [5]

Найдём предыдущую сумму оплаты клиента. Внешний SELECT вычислит разницу, используя полученный столбец prev_amount.

ПРИМЕР 10 / ИЗМЕНЕНИЕ СУММЫ ОПЛАТЫ

```
SELECT id, customer_id, amount, prev_amount,
       amount - prev_amount AS delta
FROM (
    SELECT id, customer_id, order_date, amount,
           LAG(amount) OVER (
               PARTITION BY customer_id
               ORDER BY order_date, id
           ) AS prev_amount
    FROM orders
    WHERE status = 'paid'
) AS t
ORDER BY customer_id, order_date, id;
```

Фрагмент результата: Анна

id	amount	prev_amount	delta
101	1 000	NULL	NULL
103	2 000	1 000	1 000
106	2 000	2 000	0
109	3 000	2 000	1 000

У первой оплаты Анны нет предыдущей строки, поэтому prev_amount и разница равны NULL. Между двумя заказами на 2 000 разница равна 0. «Предыдущая» здесь означает предыдущую строку, а не предыдущий календарный день.

Смещение и значение по умолчанию

LAG(amount, 2) смотрит на две строки назад. LAG(amount, 1, 0) вернёт 0, если нужной строки нет. Подставляйте 0 только когда это соответствует смыслу задачи: отсутствие предыдущей покупки не всегда означает покупку на ноль.

LEAD(amount) OVER (...) вернёт следующую сумму; у последней строки по умолчанию будет NULL. LAG и LEAD обращаются к строкам раздела по смещению, а не ограничиваются рамкой ROWS для агрегатов.

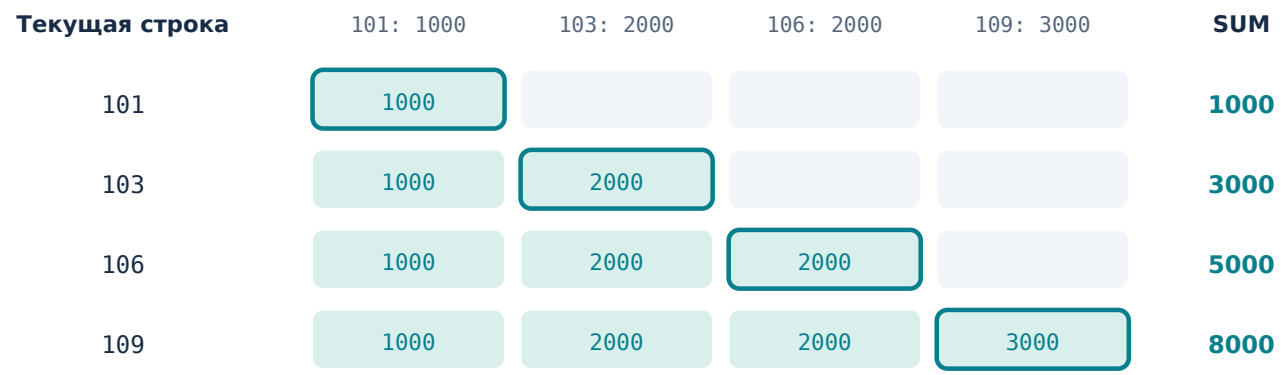
Накопительная сумма: от начала до сейчас

Рамка - это часть раздела, которую оконный агрегат использует для текущей строки. У разных строк рамка может отличаться.

ПРИМЕР 11 / НАКОПЛЕННЫЕ ОПЛАТЫ

```sql
SELECT id, customer_id, order_date, amount,
       SUM(amount) OVER (
           PARTITION BY customer_id
           ORDER BY order_date, id
           ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS running_total
FROM orders
WHERE status = 'paid'
ORDER BY customer_id, order_date, id;
```

UNBOUNDED PRECEDING - начало раздела. **CURRENT ROW** - текущая строка. **ROWS** задаёт рамку через позиции строк в указанном порядке. Здесь для каждого заказа складываем его сумму и все предыдущие оплаты клиента. [1]



Зелёным отмечены строки рамки; контур показывает текущую строку

Фрагмент результата: Анна

id	order_date	amount	running_total
101	2026-03-01	1 000	1 000
103	2026-03-03	2 000	3 000
106	2026-03-03	2 000	5 000
109	2026-03-07	3 000	8 000

Одинаковая дата не мешает: id однозначно определяет, что 103 идёт перед 106. Для другого клиента накопление начнётся заново. Если убрать ORDER BY и рамку, SUM вернёт общий итог раздела, а не накопление.

Два нюанса, которые меняют ответ

Явно задавайте рамку для накопления и используйте второй уровень запроса для фильтра по оконному результату.

1. Рамка по умолчанию учитывает равные ключи

В PostgreSQL при ORDER BY и без явной рамки используется RANGE от начала до последней строки с теми же ключами порядка, что у текущей. Если сортировать только по дате, обе оплаты Анны за 03.03 попадут в сумму вместе. [1]

Ниже - оплаты Анны. В обоих вариантах задан PARTITION BY customer_id.

id	date	amount	RANGE по дате	ROWS по дате, id
101	01.03	1 000	1 000	1 000
103	03.03	2 000	5 000	3 000
106	03.03	2 000	5 000	5 000
109	07.03	3 000	8 000	8 000

Ещё один вариант: `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW` задаёт текущую и до двух предыдущих строк. С AVG это скользящая средняя по последним трём заказам; в начале раздела строк будет меньше. Это не «за три дня».

2. Сначала вычислите номер, затем отфильтруйте

В PostgreSQL оконную функцию нельзя писать в WHERE того же уровня: WHERE логически выполняется раньше. Вложенный SELECT создаёт столбец rn, а внешний запрос уже может фильтровать его. [4]

ПРИМЕР 12 / ОДИН САМЫЙ ДОРОГОЙ ЗАКАЗ КЛИЕНТА

```
SELECT id, customer_id, amount
FROM (
    SELECT id, customer_id, amount,
           ROW_NUMBER() OVER (
               PARTITION BY customer_id
               ORDER BY amount DESC, id
           ) AS rn
    FROM orders
    WHERE status = 'paid'
) AS ranked
WHERE rn = 1
ORDER BY customer_id;
```

Результат: 109 (Анна), 110 (Борис), 108 (Вера). При равенстве сумм пример выбирает меньший id. Подзапрос в FROM можно также оформить как именованный запрос WITH ranked AS (...), затем SELECT ... FROM ranked WHERE rn = 1.

> **Порядок рассуждения**
>
> FROM / JOIN → WHERE → GROUP BY → HAVING → оконные вычисления → ORDER BY результата → LIMIT. Это упрощённая логическая модель, а не физический план выполнения.

Три задачи на оконные функции

Учитывайте только paid. Для всех последовательностей используйте order_date, id. Клиентов без оплат в результаты не включайте.

1. Последний оплаченный заказ

Для каждого платившего клиента выберите один последний заказ. Используйте ROW_NUMBER и внешний запрос. Более поздняя дата важнее, а при равенстве дат выберите больший id. Выведите customer_id, id заказа, order_date, amount; сортировка - по customer_id.

Контроль: клиент 1 - заказ 109, 07.03, 3 000; клиент 2 - 110, 08.03, 2 000; клиент 3 - 108, 06.03, 3 500. Все даты - 2026 год.

2. Две самые большие различные суммы

Для каждого клиента выведите все оплаченные заказы, суммы которых входят в **две наибольшие различные суммы** этого клиента. Используйте DENSE_RANK. Выведите customer_id, id, amount, place; сортировка - customer_id, place, id.

Контроль: 8 строк. У Анны - 109 (место 1), 103 и 106 (место 2); у Бориса - 110 (место 1), 102 и 105 (место 2); у Веры - 108 (место 1), 104 (место 2). Подумайте, почему ROW_NUMBER здесь не подходит.

3. История оплат клиента

Постройте отчёт по всем оплатам: customer_id, id, amount, running_total и delta. running_total - сумма оплат клиента от первой до текущей строки включительно; delta - разница с предыдущей оплатой. Используйте SUM с явной рамкой ROWS и LAG. Для первой оплаты delta = NULL. Сортировка - customer_id, order_date, id.

Анна / id	amount	running_total	delta
101	1 000	1 000	NULL
103	2 000	3 000	1 000
106	2 000	5 000	0
109	3 000	8 000	1 000

Контроль: всего 9 строк. Выше - фрагмент для Анны. Последний running_total у Бориса равен 4 000, у Веры - 6 000. Отменённый заказ 107 отсутствует.

САМОПРОВЕРКА

Шесть вопросов по двум урокам

Сначала ответьте своими словами, затем подтвердите ответ запросом или маленьким примером.

1. Скалярный результат

Что произойдёт в PostgreSQL, если скалярный подзапрос вернёт ноль строк? А если две строки?

2. Корреляция

По какой строке примера 4 видно, что подзапрос коррелированный? Чем его средняя отличается от средней в примере 1?

3. Логика NULL

Почему 5 NOT IN (1, 2, NULL) не даёт TRUE? Как записать проверку отсутствия связанного заказа через NOT EXISTS?

4. Группы и строки

Чем SUM(amount) с GROUP BY отличается от SUM(amount) OVER (PARTITION BY customer_id) по числу и смыслу строк?

5. Равные суммы

Какие места дадут ROW_NUMBER, RANK и DENSE_RANK для сумм 3000, 2000, 2000, 1000 по убыванию? Где нужен id для устойчивого результата?

6. Порядок и рамка

Почему номер строки фильтруют во внешнем запросе? Как рамка по умолчанию при ORDER BY order_date обрабатывает два заказа с одной датой?

Короткая памятка

Подзапрос: определите форму результата и его связь с внешней строкой. Окно: определите входные строки, раздел, порядок и рамку. Проверяйте повторы, NULL и границы разделов.

Источники для углубления

[1] Value Expressions: скалярные подзапросы и рамки окна

[2] Subquery Expressions: IN, EXISTS, NOT IN

[3] Table Expressions: подзапросы в FROM

[4] Tutorial: Window Functions

[5] Window Functions: ранги, LAG, LEAD

Официальная документация PostgreSQL. Ссылки кликабельны. Примеры и учебные данные составлены для этого конспекта.